TIME SERIES FORECASTING AND TEXT ANALYSIS USING NLP

PROJECT TITLE : RETAIL SALES TREND & SEASONALITY ANALYSIS

SUBMITTED BY:

NAME : PRIYANKA B

REG NO : 727723EUAI093

DEPT & SEC : AI&DS - B

RETAIL SALES TREND & SEASONALITY ANALYSIS

1. Introduction

Retail businesses generate large volumes of sales data over time. Understanding how sales change across weeks, seasons, and special events is essential for making informed business decisions. Time series analysis helps identify long-term trends, recurring seasonal patterns, and random fluctuations in sales data. This project focuses on performing a comprehensive statistical analysis of retail sales data using the Walmart dataset to understand its underlying structure and suitability for future forecasting.

2. Problem Statement

A nationwide retail company operates multiple outlets and records sales transactions over time. Sales fluctuate due to factors such as seasonal demand, holidays, and promotional campaigns. Management lacks clarity on:

- Whether sales show a long-term upward or downward trend
- The impact of recurring seasonal patterns such as holidays
- The amount of randomness or noise present in the data
- Whether the sales data is stationary and suitable for forecasting models

Without understanding these characteristics, future predictive models may produce unreliable results, leading to poor business decisions.

3. Project Objectives

The main objectives of this project are:

- To analyze overall and store-wise retail sales trends
- To identify and quantify seasonal patterns in sales data
- To decompose the time series into trend, seasonality, and residual components
- To analyze autocorrelation and partial autocorrelation using ACF and PACF plots
- To test stationarity of the dataset using statistical methods
- To prepare the dataset for future forecasting models

4. Dataset Description

The project uses the **Walmart Sales Dataset** obtained from Kaggle. The dataset contains weekly sales data for multiple Walmart stores.

Key Attributes:

- **Store**: Unique store identifier
- **Date**: Week of sales record
- **Weekly_Sales**: Total weekly sales for the store
- **IsHoliday**: Indicates whether the week includes a major holiday
- **Temperature, Fuel_Price, CPI, Unemployment**: Additional economic and environmental factors

The dataset spans multiple years, making it suitable for trend and seasonality analysis.

5. System Requirements

- Python 3.x
- Pandas, NumPy
- Statsmodels
- Matplotlib, Seaborn

6. Methodology

6.1 Data Preprocessing

- Loaded the dataset using Pandas
- Converted the Date column to datetime format
- Sorted data chronologically
- Aggregated weekly sales across all stores for overall analysis

6.2 Exploratory Data Analysis

- Visualized weekly sales to observe trends and fluctuations
- Identified potential seasonal spikes around holidays

6.3 Time Series Decomposition

The sales time series was decomposed into:

- **Trend**: Long-term movement in sales
- **Seasonality**: Recurring patterns observed annually
- **Residuals**: Random noise and unexplained variations

An additive decomposition model with a 52-week period was used to reflect yearly seasonality.

6.4 Autocorrelation Analysis

- **ACF (Autocorrelation Function)** was used to analyze correlation between current and past sales values
- **PACF (Partial Autocorrelation Function)** helped identify direct relationships at specific lags

These plots helped confirm the presence of seasonality and non-stationarity.

6.5 Stationarity Testing

- The Augmented Dickey-Fuller (ADF) test was applied to check stationarity
- Since the series was non-stationary, first-order differencing was applied
- Stationarity was confirmed after differencing

7. Results and Observations

- The sales data shows a clear long-term trend
- Strong seasonal patterns were observed, especially during holiday periods
- Significant autocorrelation indicated dependence on previous weeks' sales
- The original series was non-stationary but became stationary after differencing

8. Conclusion

This project successfully analyzed the Walmart retail sales data using time series statistical techniques. The analysis revealed meaningful trends and seasonal patterns, validated the presence of autocorrelation, and ensured stationarity for future forecasting. The prepared dataset is now suitable for building predictive models such as ARIMA or SARIMA.

9. Future Scope

- Build forecasting models (ARIMA/SARIMA)
- Perform store-level comparative forecasting
- Analyze the impact of economic indicators on sales
- Extend analysis to monthly or quarterly sales aggregation

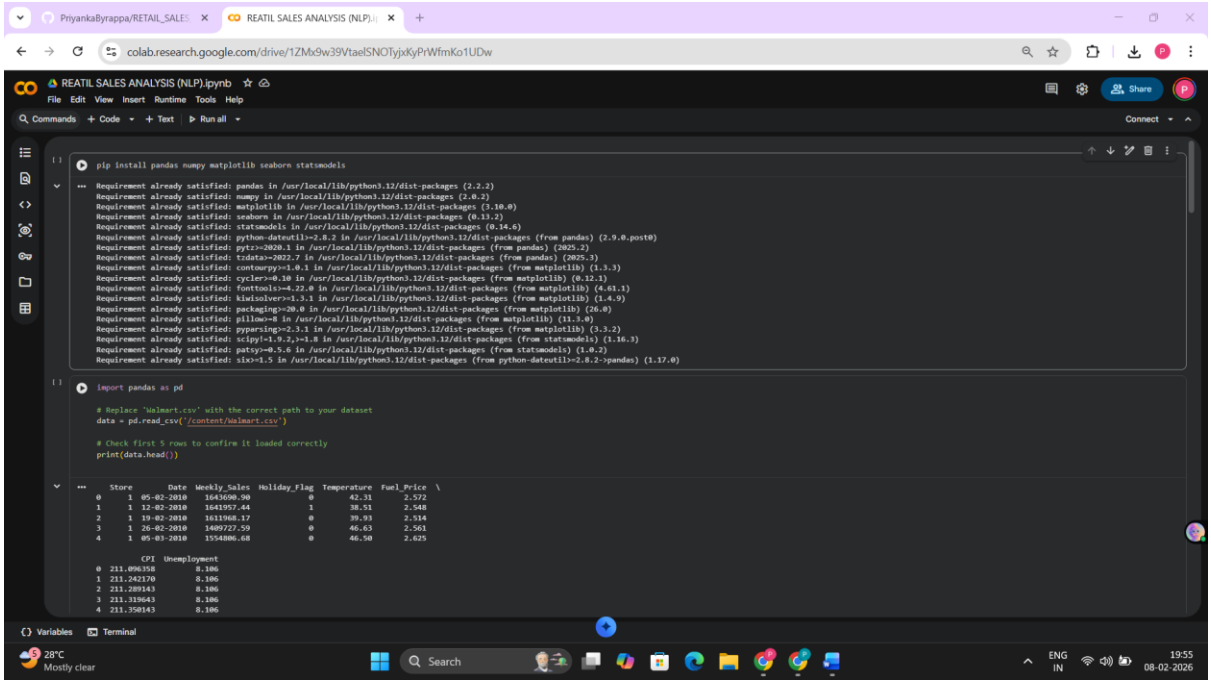
10. Tools and Technologies Used

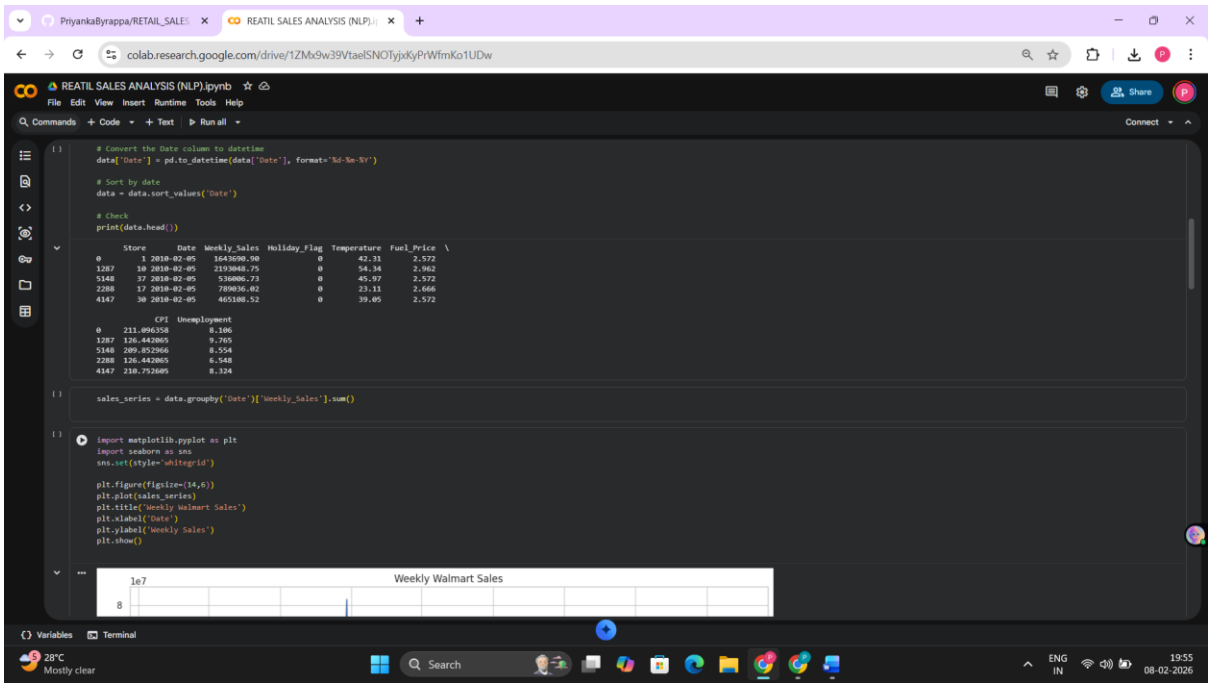
- Python
- Pandas and NumPy for data handling
- Statsmodels for time series analysis
- Matplotlib and Seaborn for visualization

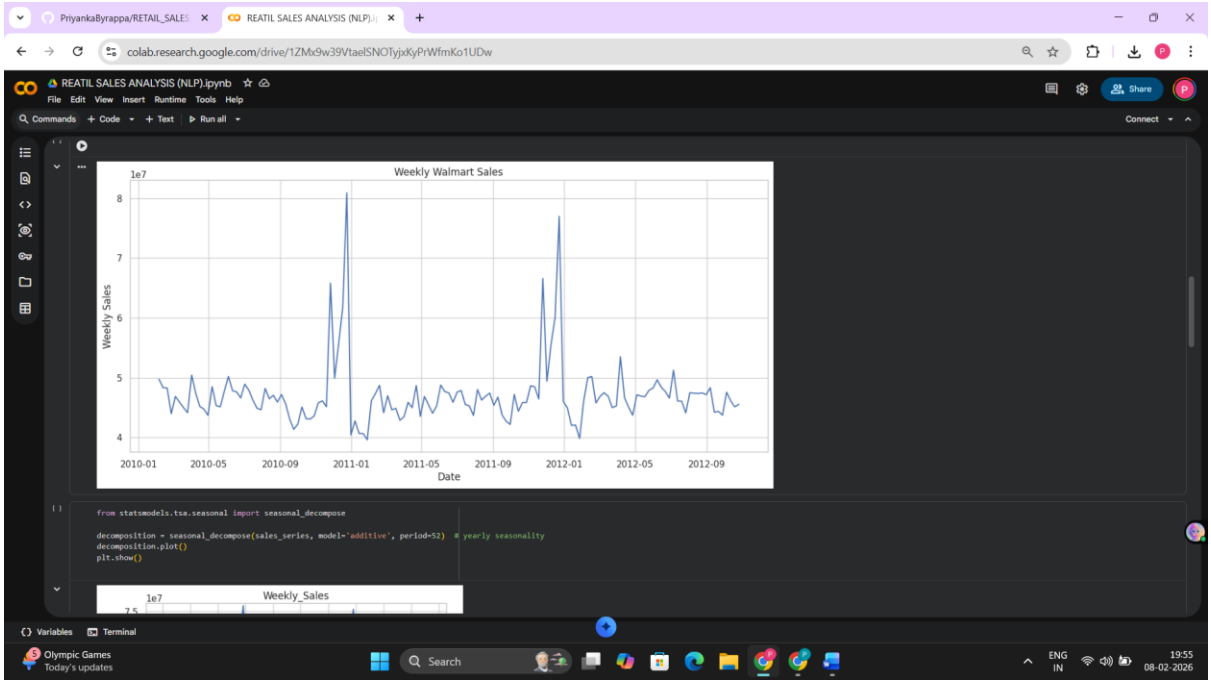
11. References

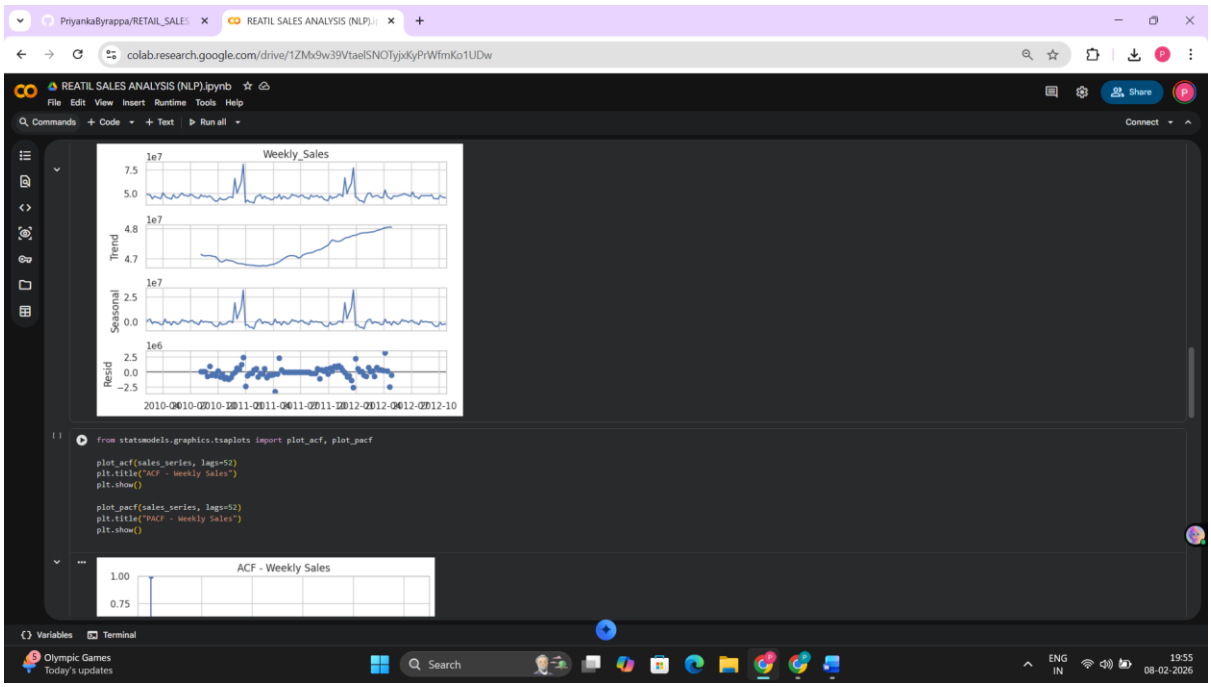
- Walmart Sales Dataset – Kaggle
- Statsmodels Documentation
- Time Series Analysis Concepts

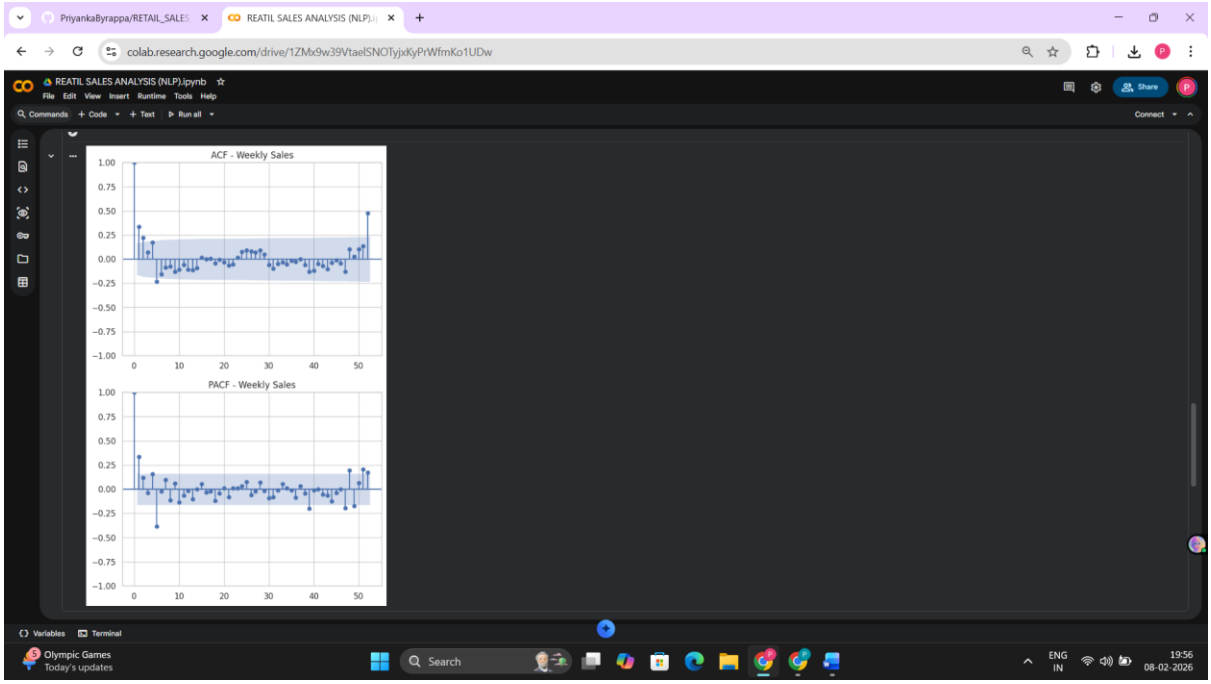
SCREENSHOTS (OUTPUT):

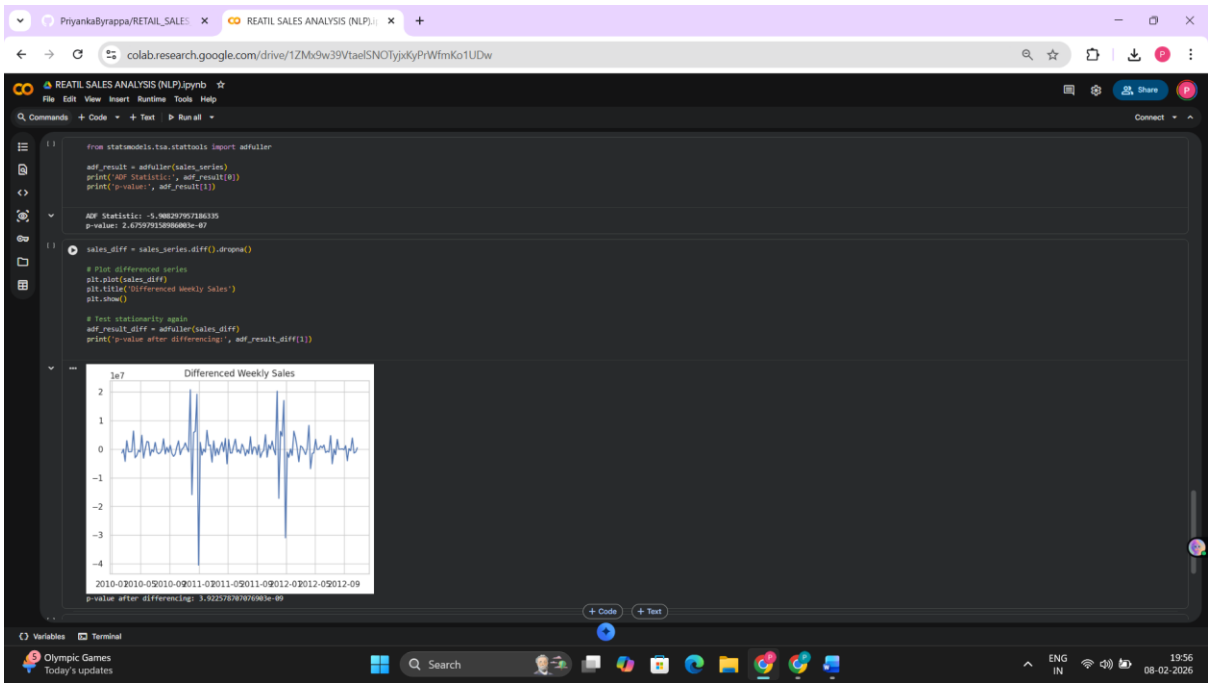





REATIL SALES ANALYSIS (NLP).ipynb
Weekly Walmart Sales
Weekly Sales
Date
from statsmodels.tsa.seasonal import seasonal_decompose
decomposition = seasonal_decompose(sales_series, model='additive', period=52) # yearly seasonality
decomposition.plot()
plt.show()
Weekly_Sales


REATIL SALES ANALYSIS (NLP).ipynb
Weekly_Sales
Trend
Seasonal
Resid
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
plot_acf(sales_series, lags=52)
plt.title("ACF - Weekly Sales")
plt.show()
plot_pacf(sales_series, lags=52)
plt.title("PACF - Weekly Sales")
plt.show()
ACF - Weekly Sales


PriyankaByrappa/RETAIL_SALES
REATIL SALES ANALYSIS (NLP).ipynb
colab.research.google.com/drive/1ZMx9w39VtaelSNOTyjxKyPrWfmKo1UDw
REATIL SALES ANALYSIS (NLP).ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Share
Connect
ACF - Weekly Sales
PACF - Weekly Sales
Variables
Terminal
Olympic Games
Today's updates
Search
ENG
IN
19:56
08-02-2026


PriyankaByrappa/RETAIL_SALES
REATIL SALES ANALYSIS (NLP).ipynb
colab.research.google.com/drive/1ZMx9w39VtaelSNOTyjxKyPrWfmKo1UDw
REATIL SALES ANALYSIS (NLP).ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Share
Connect
from statsmodels.tsa.stattools import adfuller
adf_result = adfuller(sales_series)
print('ADF Statistic:', adf_result[0])
print('p-value:', adf_result[1])
ADF Statistic: -5.908279757186335
p-value: 2.6759791585866003e-07
sales_diff = sales_series.diff().dropna()
# Plot differenced series
plt.plot(sales_diff)
plt.title('Differenced Weekly Sales')
plt.show()
# Test stationarity again
adf_result_diff = adfuller(sales_diff)
print('p-value after differencing:', adf_result_diff[1])
Differenced Weekly Sales
1e7
p-value after differencing: 3.922578787076903e-09
+ Code + Text
Variables
Terminal
Olympic Games
Today's updates
Search
ENG
IN
19:56
08-02-2026